

YUNNAN UNIVERSITY

Professional Training – YN3012170034 (Intermediate level)

Individual Mid-term Project Report

Traffic Sign Recognizer with Weather Stress Test

Robustness, calibrated confidence and a "not sure" option for traffic-sign classification under simulated bad weather

Student	AFZALI SAJAD
Student ID	20243120076
Instructor	AHMAD ZAHIR
Project folder / branch	afz076
Code repository	github.com/xahiru/PT22026, folder `afz076/` (Pull Request #20)
Date	November 2026

Abstract

Image classifiers for traffic signs reach very high accuracy on clean benchmark photographs, but a car drives in fog, rain, darkness and glare. This project asks three practical questions: (1) how quickly does the accuracy of a good traffic-sign classifier fall when the image quality gets worse; (2) does training with simulated weather make the classifier more robust, and does a bigger or pre-trained network help; and (3) can the system know when it is wrong and say “I am not sure” instead of guessing?

I built an end-to-end system on the German Traffic Sign Recognition Benchmark (GTSRB, 43 classes). Six weather and image-quality effects (fog, night, rain, blur, sensor noise and glare, each at five severities) are simulated with code written for this project. Three models are compared: a small convolutional network written from scratch (SmallCNN, 1.2 M parameters), an ImageNet-pretrained ResNet-18 (11.2 M) and an ImageNet-pretrained ConvNeXt-tiny (27.9 M). Each is trained twice, once on clean images and once with weather augmentation, giving six models. The training set is split by physical sign rather than by image, which avoids the information leak that inflates accuracy in many GTSRB experiments. Confidence is calibrated with temperature scaling, measured with the expected calibration error (ECE), and used for a reject option. Grad-CAM heatmaps show which part of the image drives a decision. Everything is available in an interactive Gradio application.

All six models reach 98.5–99.4 % accuracy on the clean test set (12,630 images), yet the clean-trained models collapse under strong weather: at the highest severity their accuracy drops to 15–34 % at night and 20–63 % with noise. Weather-augmented training raises these numbers to 64–67 % and 87–90 % without hurting clean accuracy. A surprising finding is that the small network trained from scratch with weather augmentation is as robust as the much larger pretrained models, so the training data mattered more than the model size. Glare, which was deliberately never used in training, stays very hard for every model (15–27 % at the highest severity). Temperature scaling gives no consistent improvement because the models are already well calibrated on clean data, but under bad conditions all models become over-confident, and a confidence threshold of 0.7 lets the best weather-trained models answer 92–97 % of degraded images with 97–98 % accuracy on the answered ones. The limitations of the study, mainly the simulated weather and the single training run per model, are discussed.

Keywords: traffic sign recognition, GTSRB, robustness, weather simulation, data augmentation, calibration, temperature scaling, reject option, Grad-CAM.

Contents

1. Introduction
2. Related Work
3. System Design and Methodology
4. Implementation
5. Results and Discussion
6. Conclusion and Future Work
7. References
8. Appendix A: Reproducing the Results

List of figures

1. System overview
2. Weather simulations on a schematic speed-limit sign
3. Training curves of the six models
4. Accuracy against severity for the six weather conditions
5. Reliability diagrams before and after temperature scaling
6. Coverage against accuracy for the reject option
7. The Gradio application
8. Night, severity 5: clean and weather-trained ResNet-18 on the same sign
9. The “Not sure” reject option in dense fog

1 Introduction

1.1 Motivation

Traffic signs carry the rules that drivers and driver-assistance systems must obey. Automatic recognition is therefore a standard computer-vision task, and on the best-known benchmark, the German Traffic Sign Recognition Benchmark (GTSRB) [1], modern networks exceed 99 % accuracy. But a benchmark score on well-lit, well-focused photographs says little about what happens on a foggy night or when the low sun shines straight into the camera. A system that is 99 % accurate in good conditions and silently wrong in bad ones is dangerous, because it gives the user no hint that it should not be trusted.

Two properties are therefore needed beyond raw accuracy. The first is *robustness*: accuracy should degrade gracefully when the image quality drops. The second is *honest confidence*: when the model is likely to be wrong, its confidence should be low, so that the system can ask for another image, hand control to the driver or fall back on another sensor. This project studies both on a realistic but fully reproducible task.

1.2 Problem statement and objectives

Given a cropped colour image of a single traffic sign, the system must output one of 43 sign classes, a confidence value for it, and a decision to answer or to abstain. The objectives are:

1. Build a reliable GTSRB classifier and an evaluation protocol without data leakage.
2. Simulate six realistic quality problems (fog, night, rain, blur, noise, glare) at five severities, and measure how accuracy falls.
3. Compare three architectures of different size and origin, each trained on clean data and with weather augmentation.
4. Measure how well the confidence of each model is calibrated, test temperature scaling, and evaluate a reject (“not sure”) option.
5. Explain individual predictions with Grad-CAM heatmaps.
6. Deliver everything as an interactive application and a clean, tested, documented code base submitted through a GitHub pull request.

1.3 Contributions

- A leakage-free train/validation split by physical sign (Section 3.2).
- A reproducible weather-simulation module with per-condition severity tables, and a held-out design in which glare and severity 5 are never used for training, so that they measure generalisation (Section 3.4).
- A controlled 3 × 2 comparison of architectures and training regimes under identical settings (Section 5).
- An analysis of calibration and selective classification under distribution shift, including a negative result for temperature scaling (Sections 5.5–5.6).
- An interactive Gradio application with weather sliders, a reject threshold and Grad-CAM, plus 61 automated tests.

1.4 Structure of this report

Section 2 reviews related work. Section 3 describes the data, models, weather simulation, training and evaluation methods. Section 4 explains the implementation and the software engineering practices. Section 5 presents and discusses the results, and Section 6 concludes with limitations and future work. Appendix A lists the commands to reproduce everything.

2 Related Work

2.1 Traffic sign recognition and GTSRB

Stallkamp et al. [1] introduced GTSRB for a competition at IJCNN 2011. It contains more than 50,000 images of 43 sign classes recorded on German roads, with 39,209 training and 12,630 test images in the common version. The authors measured the performance of human volunteers at 98.84 % and reported that the best machine-learning entries were already comparable. The winning approach, a multi-column committee of convolutional neural networks by Cireşan et al. [2], reached 99.46 %, better than the human average, and Sermanet and LeCun [3] showed the value of multi-scale convolutional features. Because the benchmark is nearly solved in terms of clean accuracy, current research interest has shifted to what the benchmark does not test, namely robustness and reliability, which is the focus of this project.

2.2 Convolutional architectures

Convolutional networks learn a hierarchy of filters from data. Batch normalisation [4] stabilises their training. Residual networks (ResNet) [5] add skip connections that let each block learn a correction to its input, which makes deep networks trainable. ConvNeXt [6] modernises the plain ConvNet design with larger kernels, layer normalisation and a Transformer-like layout of stages, and matches Transformer accuracy on ImageNet [7]. In this project ResNet-18 and ConvNeXt-tiny are initialised from ImageNet weights and fine-tuned, while a four-block SmallCNN is trained from scratch so that one model is small enough to be understood completely.

2.3 Robustness to image corruptions

Hendrycks and Dietterich [8] showed that networks with excellent clean accuracy can lose most of it under common corruptions such as fog, noise or blur, and proposed benchmarks that apply such corruptions at graded severities. The approach of this project follows the same idea (graded severities of named effects) but implements simpler, fully transparent simulations at the image size used for the models, and applies them to a safety-relevant domain. Training with corrupted images (data augmentation) is the standard first remedy, and it can be evaluated honestly only on corruptions that were not part of the training mixture, which motivates the held-out glare condition used here.

2.4 Calibration and temperature scaling

A classifier is *calibrated* if, among all predictions made with confidence 0.8, about 80 % are correct. Guo et al. [9] showed that modern deep networks are often over-confident and that a single parameter, a temperature T that divides the logits before the softmax, fitted on held-out data, is a simple and effective fix. The expected calibration error (ECE) summarises the gap between confidence and accuracy over confidence bins. Calibration is known to deteriorate under distribution shift, which is exactly the situation created by bad weather.

2.5 Selective classification and explainability

In selective classification [10] a model may abstain when its confidence is below a threshold. The trade-off is between *coverage* (the share of inputs answered) and the accuracy on the answered inputs. Grad-CAM [11] explains a convolutional network’s decision by weighting the activation maps of a chosen layer with the gradient of the class score, giving a coarse heatmap of the image regions that supported the prediction.

3 System Design and Methodology

3.1 Overview

Figure 1 shows the pipeline. Training and evaluation run offline on a GPU, and the interactive application loads the stored models and works on a CPU. The reject threshold and the weather simulation are applied at prediction time, which is why the application can show the effect of bad weather on any photograph that the user supplies.

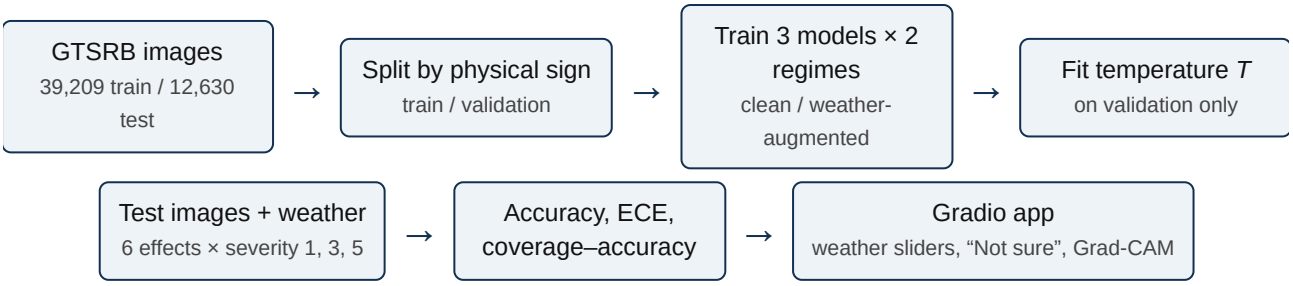


Figure 1. System overview: offline training and evaluation (top) and evaluation and application (bottom).

3.2 Dataset and leakage-free split

I use the Kaggle distribution of GTSRB, which contains a training folder with one sub-folder per class (39,209 images) and a test set of 12,630 images with labels in a CSV file. All images are converted to RGB and resized to 64×64 pixels with bilinear interpolation. Real images range from about 15×15 to 250×250 pixels, so this size keeps the detail of the larger signs while remaining cheap to train and to simulate weather on.

A subtle problem of GTSRB is that the training images are not independent. Each physical sign was photographed in a track of about 30 consecutive frames as the car approached it, so the frames of one sign are almost identical. If the images are shuffled randomly into training and validation sets, frames of the same sign end up on both sides, and the validation accuracy is inflated because the model has effectively seen the validation images. I therefore define the *group* of an image as the pair (class, track) parsed from its file name (for example `00012_00007_00021.png` belongs to sign 00012_00007) and move whole groups: 15 % of the signs of each class go to validation. This gives 32,610 training images, 6,599 validation images from 220 different physical signs, and the official 12,630 test images that are used only for the final measurements.

3.3 Models

All models end in a 43-way linear layer and take a normalised 64×64 RGB image (channel means and standard deviations estimated on GTSRB).

- **SmallCNN** (1,184,267 parameters, trained from scratch). Four blocks, each made of two 3×3 convolutions with batch normalisation and ReLU followed by 2×2 max pooling, with 32, 64, 128 and 256 channels, then global average pooling, dropout 0.3 and a linear layer. It is the baseline architecture that I can describe completely.
- **ResNet-18** (11,198,571 parameters). A residual network [5] with ImageNet weights from torchvision; the final layer is replaced by a new 43-way layer and the whole network is fine-tuned.
- **ConvNeXt-tiny** (27,853,195 parameters). A modern convolutional network [6] with ImageNet weights, fine-tuned in the same way.

Replacing the last layer is necessary because ImageNet has 1,000 classes, not 43. Using pretrained weights tests whether knowledge learned from natural images helps robustness, in comparison with the from-scratch SmallCNN.

3.4 Weather and image-quality simulation

The module `corruptions.py` implements six effects. Each takes an 8-bit RGB image and a severity from 1 (mild) to 5 (extreme), and uses a seeded random generator so that results are reproducible. Figure 2 illustrates them on a schematic sign.

- **Fog**: blends the image with a smooth, light-grey haze layer; the blending weight grows from 0.20 to 0.80.
- **Night**: applies a gamma curve (exponent 1.4) and scales brightness from 0.70 down to 0.18, then adds Gaussian sensor noise (standard deviation 4 to 20 grey levels).
- **Rain**: draws 10 to 70 semi-transparent diagonal streaks and slightly dims the scene.
- **Blur**: Gaussian blur with radius 0.5 to 2.2 pixels (out-of-focus or moving camera).

- **Noise**: additive Gaussian noise with standard deviation 8 to 50 grey levels (poor sensor).
- **Glare**: adds a bright radial blob at a random position, with relative radius 0.25 to 0.70 and strength 0.40 to 1.00 (low sun or headlights).

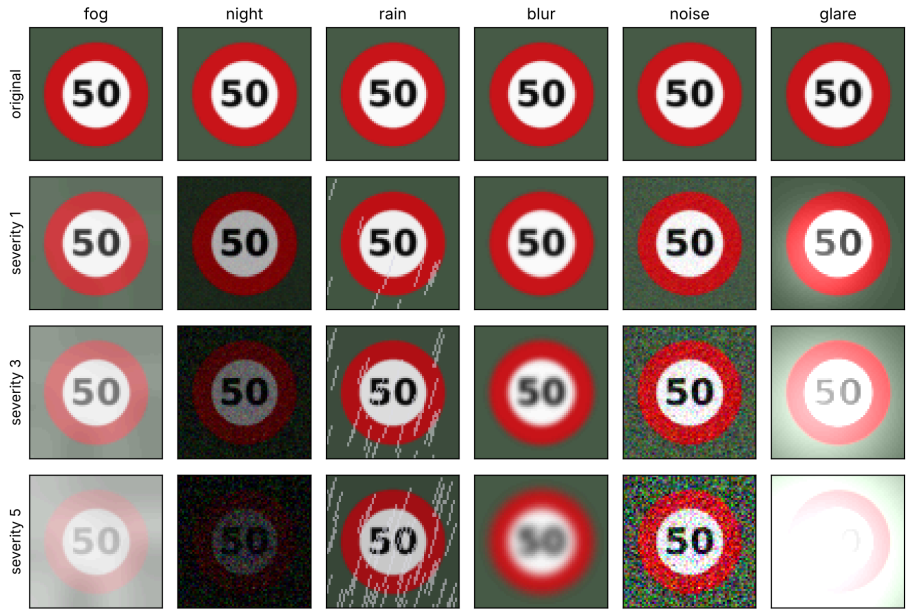


Figure 2. The six simulated effects at severities 1, 3 and 5. The sign is a schematic speed-limit sign drawn for illustration only, not a GTSRB image. At severity 5 the glare removes almost all information from the image.

Held-out design. During weather training each image is, with probability 0.5, corrupted by one randomly chosen condition from {fog, night, rain, blur, noise} at a random severity from 1 to 4. *Glare and severity 5 are never used in training.* Evaluating on them therefore measures generalisation to stronger and to unseen conditions, and not memorisation of the training distribution.

3.5 Training procedure

All six models use the same recipe, so that differences come from the architecture and the data and not from tuning: AdamW optimiser [12] with weight decay 10^{-4}, the one-cycle learning-rate schedule [13], batch size 128, mixed-precision arithmetic on the GPU, and a fixed random seed (42). The loss is the plain cross-entropy without label smoothing, because label smoothing changes the confidence of the model and would hide the calibration effects that I want to study. Every training image receives a small geometric jitter (rotation of ±10° and a shift of ±4 pixels). **No horizontal flip is used**, since flipping turns “turn left” into “turn right” and would corrupt the labels. The checkpoint with the best accuracy on the *clean* validation set is kept. Table 1 lists the settings.

Table 1. Training settings (the number of epochs and the peak learning rate differ by model; the rest is shared).

Model	Parameters	Initial weights	Epochs	Peak learning rate
SmallCNN	1,184,267	random	15	2×10^{-3}
ResNet-18	11,198,571	ImageNet	8	1×10^{-3}
ConvNeXt-tiny	27,853,195	ImageNet	6	5×10^{-4}

The weather-trained and clean-trained versions of a model differ only in the weather probability (0.5 versus 0). The training of one model took between about 3 and 7 minutes on a cloud GPU.

3.6 Calibration and temperature scaling

For each model the softmax probabilities are computed from the logits z as softmax(z/T). The temperature T is found by minimising the negative log-likelihood on the clean **validation** set with the L-BFGS optimiser (the parameter being optimised is log T so that T stays positive). The test set is never used for fitting. A value $T > 1$ softens the probabilities (the raw model was over-confident) and $T < 1$ sharpens them. Calibration is measured by the expected calibration error with 15 equal-width bins,

$$ECE = \Sigma_b \, (n_b/n) \cdot | \, accuracy(b) - confidence(b) \, |,$$

where n_b is the number of predictions in bin b. Lower is better, and 0 means perfect calibration.

3.7 Reject option

The application and the evaluation use a simple rule: if the calibrated confidence of the top class is below a threshold (default 0.70), the system answers “Not sure”. For a set of thresholds I report the *coverage* (share of images answered) and the *accuracy on the answered images*. Rejected images are not counted as errors, so an ideal rejector would raise the accuracy of the answered images while keeping the coverage as high as possible.

3.8 Grad-CAM

Grad-CAM [11] is implemented in 20 lines of PyTorch. A forward hook stores the activations A of a target layer and the gradient of the class score with respect to them. The importance of each channel is the spatial mean of its gradient, the heatmap is ReLU($\Sigma_k \, w_k \, A_k$), and it is upsampled to the image size and normalised to [0, 1]. With 64×64 inputs the deepest stages have only 2×2 feature maps, which would produce a uselessly coarse heatmap, so I explain earlier layers that still have 4×4 or 8×8 maps: the last convolution of SmallCNN, `layer3` of ResNet-18 and stage 3 of ConvNeXt-tiny.

3.9 Evaluation protocol

Every model is evaluated on the complete official test set (12,630 images) in the clean condition and for each of the six effects at severities 1, 3 and 5, which gives 19 test conditions per model. The simulation of each test image is seeded by the image index, so every model sees exactly the same corrupted images. I report accuracy, ECE before and after temperature scaling, and the coverage–accuracy trade-off. Each configuration was trained once; the consequences of this are discussed in Section 5.8.

4 Implementation

4.1 Technology and project layout

The code is written in Python 3.11 with PyTorch and torchvision [14] for the models, NumPy and Pillow for image handling, pandas and Matplotlib for the evaluation, and Gradio [15] for the user interface. Pretrained weights come from torchvision. The project lives in the folder `afz076/` of the course repository (Table 2). The data set, the trained weights and all other large files are *not* stored in the repository, as required; the six trained models are shared through a Google Drive link given in the README.

Table 2. Modules of the project.

File	Purpose
`src/config.py`, `src/classes.py`	Central settings (image size, conditions, severities, seed) and the 43 class names
`src/data.py`	Loading GTSRB, split by physical sign, PyTorch dataset with geometric jitter and weather augmentation
`src/corruptions.py`	The six weather and quality simulations with five severities each
`src/models.py`	SmallCNN, ResNet-18 and ConvNeXt-tiny with 43-way heads; Grad-CAM target layers
`src/trainer.py`	Training loop (AdamW, one-cycle, mixed precision), checkpoint saving and loading, batch inference
`src/calibration.py`	ECE, reliability bins, temperature fitting, selective accuracy
`src/gradcam.py`, `src/predict.py`	Grad-CAM and single-image prediction with confidence and reject decision
`training/train.py`	Command-line script to train one model
`evaluation/evaluate.py`, `make_plots.py`, `make_table.py`	Evaluation of checkpoints, figures and the Markdown results table
`app.py`	The interactive Gradio application
`tests/`	61 automated tests (pytest)
`notebooks/SignGuard_Colab.ipynb`	Notebook that runs the whole pipeline on a free Colab GPU

4.2 Design decisions

- **Separation of concerns.** The training loop and the inference helpers live in `trainer.py`, and the command-line wrapper is a thin script, so the logic can be unit-tested without running a full training.
- **Everything is configurable in one place.** Image size, conditions, severity range, seed and the default reject threshold are in `config.py`, and the training and the app read the same values, which avoids silent mismatches.
- **Reproducibility.** The data split, the weather simulation and the evaluation use fixed seeds, and the evaluation stores its numbers in `robustness.csv` and one JSON file per model, from which the figures and the README table are generated by script instead of copied by hand.
- **Temperature stored with the model.** The fitted temperature is saved inside the checkpoint, so the application automatically shows calibrated confidences.
- **Graceful failure.** The application catches errors (for example a missing model) and shows a readable message instead of crashing.

4.3 The application

The Gradio application (Figure 3) lets the user upload a picture of a sign, choose one of the six models, optionally add one of the six weather effects with a severity slider, and set the “Not sure” threshold. It shows the 64×64 image that the model actually sees, the verdict (the sign name with its confidence, or “Not sure” with the best guess), the top three probabilities and the Grad-CAM overlay. It runs on a laptop CPU and finds the models automatically in the `outputs/` folder.

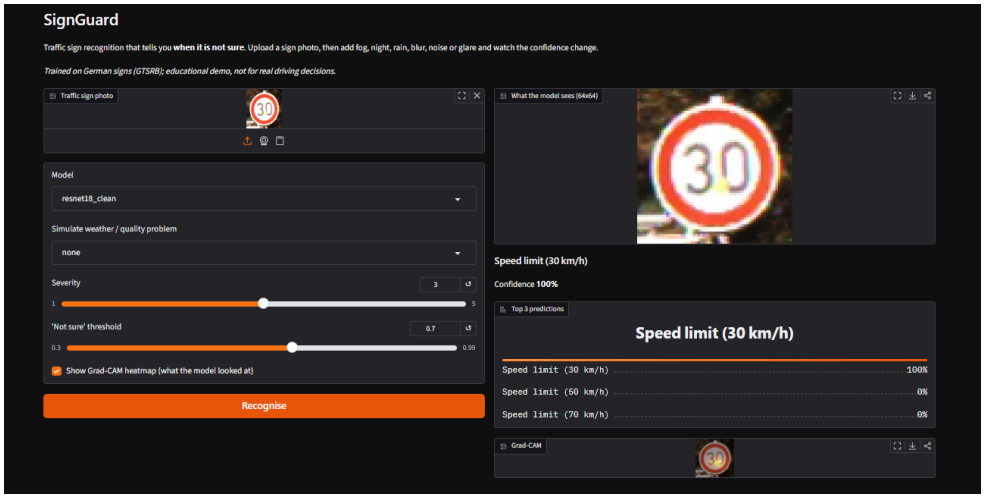


Figure 3. The Gradio application: model choice, weather simulation, reject threshold, prediction and Grad-CAM heatmap.

4.4 Testing and software quality

The test suite contains 61 pytest tests, which all pass. They check, among other things, that every corruption returns an image of the same shape and type, is deterministic for a fixed seed and gets stronger with severity; that the split by sign leaks no sign between training and validation; that the weather augmentation used in training never contains glare or severity 5; that every model produces a forward pass and a Grad-CAM map; that the ECE separates a calibrated from an over-confident example; that temperature scaling fixes over-confident logits; that the reject option trades coverage for accuracy; and that a tiny end-to-end run of training, evaluation and prediction works. All functions have docstrings, and the work was committed to Git in 21 small commits with descriptive messages and submitted as a pull request from the branch `afz076`, never to the main branch.

5 Results and Discussion

5.1 Training behaviour

Figure 4 shows the training loss and the clean validation accuracy. The pretrained networks start from a high validation accuracy (88–98 % after the first epoch) and converge in a few epochs. The SmallCNN needs more epochs, starts at about 51 % (42 % with weather augmentation) and its validation accuracy fluctuates during the high-learning-rate phase, but it ends at 99.4–99.6 %, close to the best pretrained models. The best validation accuracies were 99.4 % (SmallCNN clean), 99.6 % (SmallCNN weather), 99.0 % (ResNet-18 clean), 99.6 % (ConvNeXt-tiny clean) and 99.9 % (ConvNeXt-tiny weather). Weather-augmented models have a higher training loss, as expected, because half of their training images are degraded.

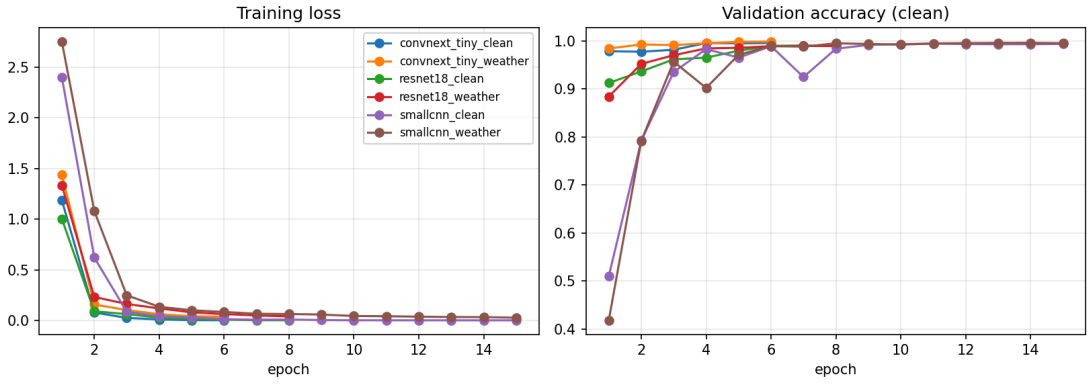


Figure 4. Training loss (left) and clean validation accuracy (right) of the six models.

5.2 Accuracy on clean images

On the clean test set all six models are within one percentage point of each other (first column of Table 3): 98.5 % (ResNet-18, clean) up to 99.4 % (ConvNeXt-tiny, weather). For reference, human volunteers reached 98.84 % in the original benchmark [1]. The differences of a few tenths of a percent are within what a different random seed could change, so on clean data the models are effectively equivalent. Importantly, **weather-augmented training changed clean accuracy by at most 0.3 points, and not in a consistent direction**: 98.9 versus 99.2 % for SmallCNN, 98.8 versus 98.5 % for ResNet-18 and 99.4 versus 99.3 % for ConvNeXt-tiny (weather versus clean). Clean accuracy alone therefore cannot tell the models apart, which is exactly why a stress test is needed.

5.3 Robustness to weather

Table 3 and Figure 5 show what happens under the strongest degradation (severity 5, which was never seen in training). Table 4 gives the intermediate severity 3.

Table 3. Test accuracy (%) on clean images and at severity 5. Glare was never used in training.

Model	Clean	Fog	Night	Rain	Blur	Noise	Glare
SmallCNN (clean)	99.2	65.8	14.8	55.9	95.8	20.2	15.0
SmallCNN (weather)	98.9	92.9	67.1	96.5	97.2	89.7	24.5
ResNet-18 (clean)	98.5	59.1	25.6	40.9	92.2	41.0	15.8
ResNet-18 (weather)	98.8	89.6	64.1	90.0	96.1	86.9	23.0
ConvNeXt-tiny (clean)	99.3	82.8	33.8	67.6	95.5	63.4	20.4
ConvNeXt-tiny (weather)	99.4	94.1	66.6	94.8	97.2	88.9	27.0

Table 4. Test accuracy (%) on clean images and at severity 3.

Model	Clean	Fog	Night	Rain	Blur	Noise	Glare
SmallCNN (clean)	99.2	94.4	63.2	79.2	99.4	61.2	85.4
SmallCNN (weather)	98.9	98.4	91.8	98.5	98.9	96.9	89.9
ResNet-18 (clean)	98.5	93.6	62.0	55.6	98.3	71.0	81.1
ResNet-18 (weather)	98.8	97.8	89.5	94.7	98.7	95.3	88.0
ConvNeXt-tiny (clean)	99.3	97.5	70.8	86.2	99.1	84.3	86.8
ConvNeXt-tiny (weather)	99.4	98.7	90.5	98.0	99.1	96.0	88.5

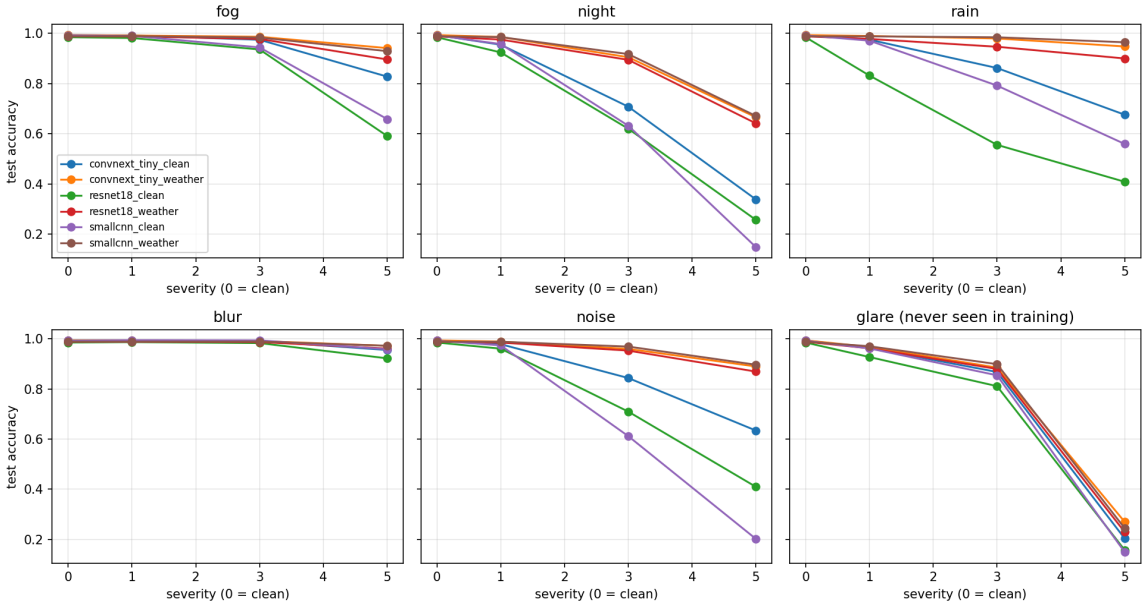


Figure 5. Accuracy as a function of severity (0 = clean) for each condition and model. Weather-trained models are the upper curves in fog, night, rain and noise.

Four observations stand out.

1. **Clean-trained models are fragile.** At severity 5, the accuracy of the clean-trained models is only 15–34 % at night, 20–63 % with noise, 41–68 % in rain and 59–83 % in fog, although they score 98.5–99.3 % on clean images. The decline is already clear at severity 3 (Table 4).
2. **Weather augmentation helps a lot, even beyond the training range.** The weather-trained models reach 64–67 % at night, 87–90 % with noise, 90–97 % in rain and 90–94 % in fog at severity 5, although training only used severities 1 to 4. For example, the SmallCNN improves from 14.8 to 67.1 % at night and from 20.2 to 89.7 % with noise.
3. **Blur is the mildest condition.** Even clean-trained models keep 92–96 % at severity 5. At 64×64 pixels the shape and colours of a sign survive moderate blur, so the learned features are not very sensitive to it.
4. **Glare is a failure for all models.** At severity 5 the accuracy is only 15–27 % for every model, including the weather-trained ones, whose gain on glare is only about 7 to 10 points. Figure 2 explains why: at severity 5 the simulated glare washes out almost the whole sign, so the information is largely *gone* from the image. This is a limit of the data and not just of the model, and it is a good example of a case where the system should abstain instead of answering (Section 5.6). Because glare is the unseen condition, the result also shows that augmentation with five effects does not automatically generalise to a sixth.

5.4 Model size and pretraining

Among the clean-trained models, ConvNeXt-tiny is clearly the most robust (for example 82.8 % in fog and 63.4 % with noise, against 59.1 and 41.0 % for ResNet-18), so a larger, more modern pretrained network does give some robustness for free. ResNet-18, however, is not consistently better than the much smaller SmallCNN: it is better at night (25.6 versus 14.8 %) and with noise (41.0 versus 20.2 %), but worse in fog (59.1 versus 65.8 %), rain (40.9 versus 55.9 %) and blur (92.2 versus 95.8 %), so pretraining is not a guarantee of robustness.

The most interesting result appears when weather augmentation is used. Then the three models are close: at severity 5 the SmallCNN reaches 92.9 % in fog and 96.5 % in rain, ConvNeXt-tiny reaches 94.1 and 94.8 %, and ResNet-18 89.6 and 90.0 %. The 1.2-million-parameter network written from scratch is as robust as, and in several conditions more robust than, the pretrained models that are 10 to 20 times larger. For this task, **what the model was trained on mattered more than how large it is**. A caution is needed: each model was trained once, so differences of one or two points between weather-trained models (for instance in fog or in rain) are within the likely run-to-run noise, and I do not claim that one of them is better.

5.5 Calibration and temperature scaling

Table 5 shows the fitted temperatures and the ECE on the clean test set. All six models are already very well calibrated on clean data (ECE below 0.007 before scaling). Temperature scaling reduced the ECE for three models (ResNet-18 clean and weather, ConvNeXt-tiny clean) and increased it for the other three, but all changes are smaller than 0.004, which is negligible. The fitted temperatures lie between 0.75 and 1.35. The value for ConvNeXt-tiny (weather), $T = 0.75$, means that this model was slightly *under*-confident on validation data, which is plausible because half of its training images were degraded and it learned to hedge. Temperature scaling is therefore not the main effect in this project; this is an honest negative result that I report, because in the literature it matters for networks that overfit and are over-confident [9], and my models, trained for few epochs with weight decay, were not strongly over-confident on clean data.

Table 5. Fitted temperature and expected calibration error on the clean test set, before and after temperature scaling.

Model	Fitted T	ECE before	ECE after
SmallCNN (clean)	1.23	0.0018	0.0055
SmallCNN (weather)	0.97	0.0018	0.0022
ResNet-18 (clean)	1.17	0.0064	0.0041
ResNet-18 (weather)	1.35	0.0043	0.0035
ConvNeXt-tiny (clean)	1.18	0.0041	0.0028
ConvNeXt-tiny (weather)	0.75	0.0029	0.0043

The picture changes under distribution shift. Table 6 gives the ECE at severity 5 for three conditions (raw / after scaling). The clean-trained models become strongly over-confident: for example the raw ECE of ResNet-18 (clean) is 0.457 at night, meaning that its stated confidence and its real accuracy differ on average by 46 points. Weather-trained models are far better calibrated in fog (raw ECE 0.007 to 0.027) and reasonably so at night (0.03 to 0.08), but they too are badly calibrated on glare (0.22–0.27 raw). A temperature fitted on clean validation images improves the ECE of the models with $T > 1$ in most of the shifted cases, by up to about 0.09, but does not repair it. It makes the two models with $T < 1$ slightly or clearly worse (ConvNeXt-tiny weather: 0.064 to 0.118 at night), because a temperature below 1 sharpens predictions that are already wrong. The reliability diagrams for all models on clean data are shown in Figure 6 (the noisy points at low confidence come from bins that contain only a few images).

Table 6. ECE at severity 5 (before / after temperature scaling) for three conditions. Lower is better.

Model	Fog 5	Night 5	Glare 5
SmallCNN (clean)	0.182 / 0.137	0.248 / 0.188	0.517 / 0.430
SmallCNN (weather)	0.007 / 0.006	0.032 / 0.038	0.217 / 0.227
ResNet-18 (clean)	0.203 / 0.157	0.457 / 0.370	0.404 / 0.323
ResNet-18 (weather)	0.017 / 0.020	0.080 / 0.015	0.274 / 0.166
ConvNeXt-tiny (clean)	0.120 / 0.105	0.112 / 0.080	0.400 / 0.352
ConvNeXt-tiny (weather)	0.027 / 0.037	0.064 / 0.118	0.274 / 0.363

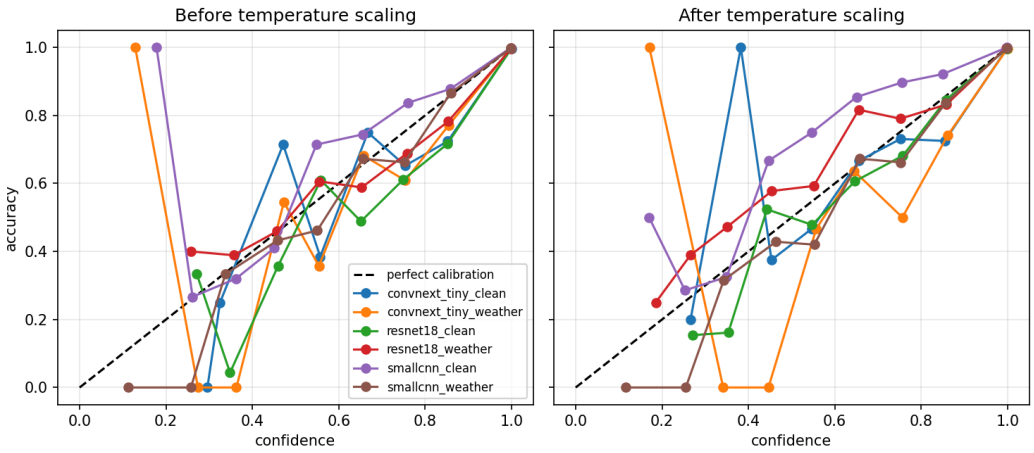


Figure 6. Reliability diagrams on the clean test set before (left) and after (right) temperature scaling. The dashed line is perfect calibration. Low-confidence bins contain very few images and are therefore noisy.

The practical conclusion is that a calibration fitted on clean data should not be trusted in bad weather. A better approach would be to fit the temperature (or another calibrator) on a validation set that contains weather effects, which is left for future work.

5.6 The reject option

Table 7 and Figure 7 show the effect of answering only when the calibrated confidence is at least 0.7. On clean images, rejecting costs almost nothing: the models still answer 98.3–99.7 % of the images and the accuracy on the answered images is 99.3–99.8 %. On the mixed severity-3 images (all six conditions pooled), the benefit is large for the clean-trained models. For the ResNet-18 (clean) the accuracy of 76.9 % if it always answers becomes 91.8 % on the 77.7 % of the images that it does answer; for the SmallCNN (clean) 80.5 % becomes 97.0 % at 75.6 % coverage. In other words, the confidence signal really is informative: the model is less sure on exactly the images where it makes errors.

Table 7. Reject option with threshold 0.7 (accuracy and coverage in %). “Sev. 3” pools all six conditions at severity 3.

Model	Sev. 3: acc. if always answering	Sev. 3: coverage	Sev. 3: acc. on answered	Clean: coverage	Clean: acc. on answered
SmallCNN (clean)	80.5	75.6	97.0	98.4	99.8
SmallCNN (weather)	95.7	95.1	98.4	98.7	99.5
ResNet-18 (clean)	76.9	77.7	91.8	98.5	99.3
ResNet-18 (weather)	94.0	92.4	98.0	98.3	99.5
ConvNeXt-tiny (clean)	87.4	89.8	94.9	99.6	99.5
ConvNeXt-tiny (weather)	95.1	97.3	96.8	99.7	99.5

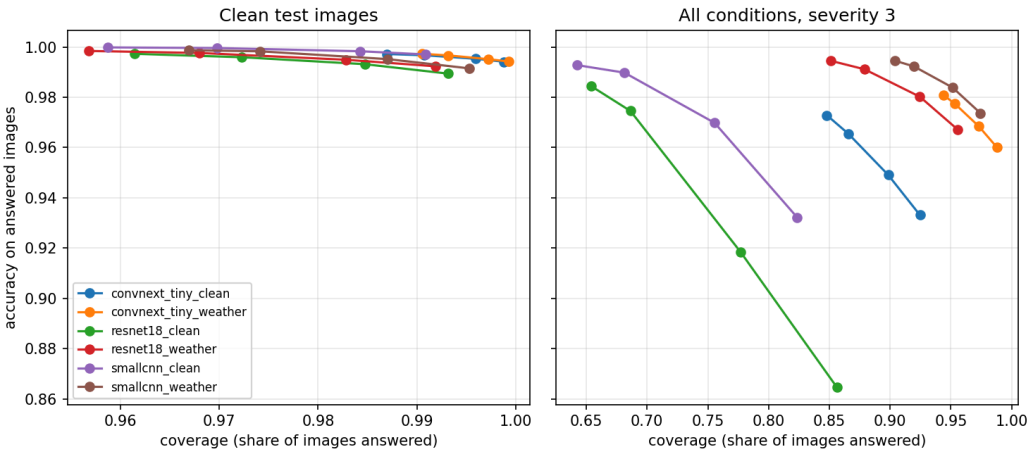


Figure 7. Coverage against accuracy on the answered images for thresholds 0.5, 0.7, 0.9 and 0.95 on clean images (left) and on pooled severity-3 images (right). Curves toward the upper right are better.

The weather-trained models combine both advantages. They answer 92–97 % of the severity-3 images with 97–98 % accuracy (SmallCNN 95.1 % at 98.4 %, ResNet-18 92.4 % at 98.0 %, ConvNeXt-tiny 97.3 % at 96.8 %). There is a trade-off between the two kinds of improvement: robust training moves a model to the upper right of Figure 7 (more answers and more accuracy), whereas rejection moves it along a curve (fewer answers, higher accuracy). A safety system will want both, and the user can choose the operating point with the threshold slider of the application. Because calibration degrades in the most extreme conditions (Section 5.5, ECE of 0.22 to 0.52 for glare at severity 5), a confidence threshold remains a useful but not a perfect safety net.

5.7 The application in use: Grad-CAM and the reject option

Grad-CAM is used as a qualitative tool. For every prediction the application overlays the heatmap on the image (Figure 3), so that a user can check where the model looked. I did not quantify the heatmaps, so they are an illustration and not a measured result, and with 64×64 inputs the maps are necessarily coarse (4×4 or 8×8 before up-sampling).

Figure 8 shows the same traffic sign (“Turn right ahead”) at night with severity 5, once with the clean-trained and once with the weather-trained ResNet-18. The image that the model sees is almost black noise, yet both models name the sign correctly, with 91 % and 100 % confidence. This is a single easy example and not a general result: Table 3 shows that the clean ResNet-18 recognises only 25.6 % of all test images at this condition. In both cases the heatmap is one blob near the centre of the image; with so little signal left I do not draw further conclusions from it.

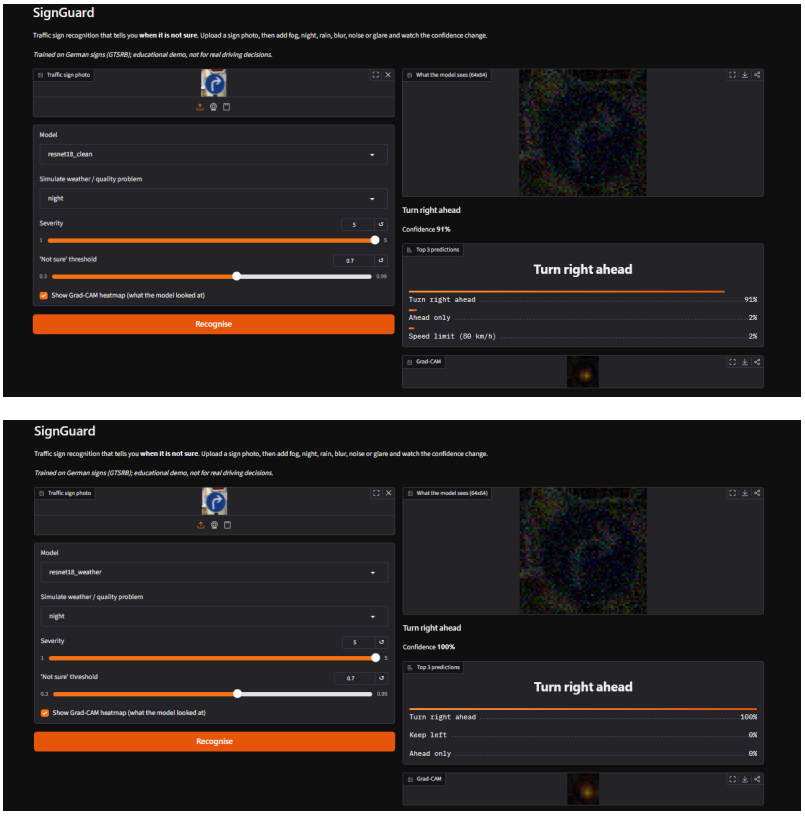


Figure 8. The same sign at night, severity 5. Top: ResNet-18 trained on clean images (“Turn right ahead”, 91 %). Bottom: ResNet-18 trained with weather augmentation (100 %).

Figure 9 shows the reject option at work. In dense fog (severity 5) the clean-trained ResNet-18 cannot read the number on a speed-limit sign. Its best guess (80 km/h) gets only 34 % confidence, and the next candidates are other speed limits (30 and 70 km/h with 27 and 11 %), which shows that the model recognises the kind of sign but not its number. Since 34 % is below the 0.7 threshold, the application answers “Not sure” and asks for a better image instead of risking a wrong speed limit.

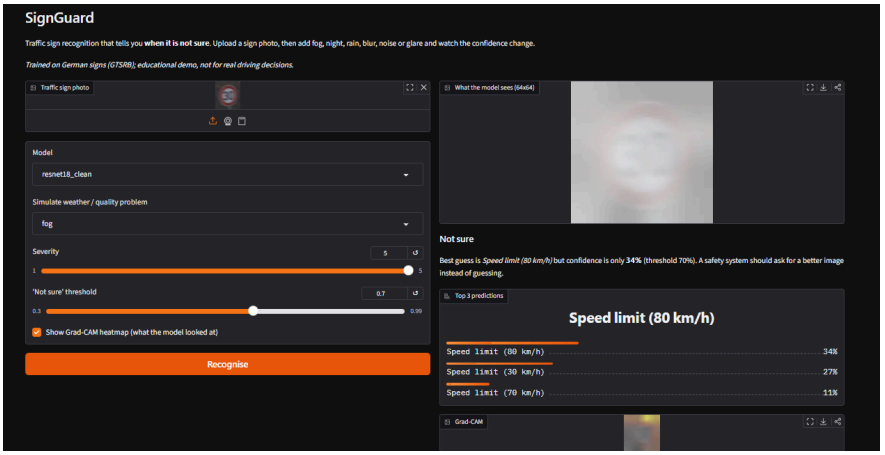


Figure 9. Reject option: clean-trained ResNet-18, fog severity 5, threshold 0.7. Confidence 34 %, so the answer is “Not sure”.

5.8 Limitations and threats to validity

- **Simulated weather.** The effects are image-processing approximations, applied after resizing, and not photographs taken in real fog or rain. Real conditions combine effects, change the scene and not just the pixels, and may behave differently. Real-world performance was therefore *not tested*, and the numbers in this report must not be read as the accuracy that the system would have on a car.

- **One run per model.** No confidence intervals could be computed. The large differences (tens of points) between clean and weather training are certainly real, but differences of one or two points between models are not reliable.
- **Cropped signs only.** The system classifies an already cropped sign, as in GTSRB. It does not detect signs in a street scene.
- **German signs only** and **64×64 input**, which loses the detail of larger images.
- **Held-out glare** was chosen to measure generalisation, but it also means that glare robustness was never attempted; a model trained with glare would probably do better.
- **Not for driving.** The project is educational and must not be used for real driving decisions.

6 Conclusion and Future Work

This project built and evaluated a traffic-sign recognizer that is tested under simulated bad weather, reports calibrated confidence and can say “Not sure”. The main findings are:

1. High clean accuracy (98.5–99.4 %) hides large weaknesses: clean-trained models lose most of their accuracy under strong night, noise, rain or fog.
2. Training with simulated weather greatly improves robustness (for example night at severity 5 from 15–34 % to 64–67 %) and does not hurt clean accuracy, even for severities that were never used in training.
3. A small network trained from scratch with weather augmentation is as robust as pretrained networks that are 10 to 20 times larger, so the training data matter more than the model size here.
4. Glare, an effect not used in training, remains very hard for all models, and is a case where the image information is simply destroyed.
5. The models are well calibrated on clean data, so temperature scaling brings no consistent gain, but under shift they become over-confident. A confidence threshold nevertheless raises the accuracy of answered images substantially, and weather-trained models can answer more than 92 % of degraded images with 97–98 % accuracy.

Future work should (a) test on real adverse-weather photographs and videos, for example from a driving data set recorded in rain and at night, (b) repeat each training with several seeds to report confidence intervals, (c) include glare in the augmentation and test truly unseen conditions, (d) fit the calibration on weather-degraded validation data, or use methods that estimate uncertainty without a fixed temperature, such as ensembles, (e) work on larger inputs and add a detector so that signs are found in full street scenes, and (f) quantify the heatmaps, for instance by measuring how much of the Grad-CAM mass falls on the sign.

References

1. J. Stallkamp, M. Schlipsing, J. Salmen and C. Igel, “Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition,” *Neural Networks*, vol. 32, pp. 323–332, 2012.
2. D. Cireşan, U. Meier, J. Masci and J. Schmidhuber, “Multi-column deep neural network for traffic sign classification,” *Neural Networks*, vol. 32, pp. 333–338, 2012.
3. P. Sermanet and Y. LeCun, “Traffic sign recognition with multi-scale convolutional networks,” in *Proc. International Joint Conference on Neural Networks (IJCNN)*, 2011.
4. S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proc. ICML*, 2015.
5. K. He, X. Zhang, S. Ren and J. Sun, “Deep residual learning for image recognition,” in *Proc. CVPR*, 2016.
6. Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell and S. Xie, “A ConvNet for the 2020s,” in *Proc. CVPR*, 2022.
7. J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li and L. Fei-Fei, “ImageNet: A large-scale hierarchical image database,” in *Proc. CVPR*, 2009.
8. D. Hendrycks and T. Dietterich, “Benchmarking neural network robustness to common corruptions and perturbations,” in *Proc. ICLR*, 2019.
9. C. Guo, G. Pleiss, Y. Sun and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proc. ICML*, 2017.
10. Y. Geifman and R. El-Yaniv, “Selective classification for deep neural networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
11. R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh and D. Batra, “Grad-CAM: Visual explanations from deep networks via gradient-based localization,” in *Proc. ICCV*, 2017.
12. I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. ICLR*, 2019.
13. L. N. Smith and N. Topin, “Super-convergence: Very fast training of neural networks using large learning rates,” in *Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications*, Proc. SPIE 11006, 2019.
14. A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
15. A. Abid, A. Abdalla, A. Abid, D. Khan, A. Alfozan and J. Zou, “Gradio: Hassle-free sharing and testing of ML models in the wild,” arXiv:1906.02569, 2019.

Appendix A Reproducing the Results

Install and run the application (the trained models are downloaded from the Google Drive link in the README into the folder `outputs/`):

```
git clone https://github.com/xahiru/PT22026.git
cd PT22026/afz076
python -m venv .venv
source .venv/Scripts/activate     # Windows Git Bash;  Linux/macOS: source
.venv/bin/activate
pip install -r requirements.txt
python app.py
```

Train one model, evaluate all six and create the figures and the results table (a GPU is recommended; the Colab notebook in `notebooks/` runs all steps):

```
python training/train.py --data-root GTSRB --model resnet18 --weather-p 0   --name
resnet18_clean
python training/train.py --data-root GTSRB --model resnet18 --weather-p 0.5 --name
resnet18_weather
python evaluation/evaluate.py --data-root GTSRB --ckpt outputs/*/model.pt
python evaluation/make_plots.py
python evaluation/make_table.py
pytest -q                         # 61 tests
```

If the application stops with a 502 error on a computer that uses a proxy or VPN, run `export NO_PROXY=127.0.0.1,localhost` before `python app.py`.